



Bloc 4

Maintenir l'application logicielle en condition opérationnelle

Dossier écrit - Projet Spity

	Expert en développement logiciel
	Dorian Joly
	Ynov 2024 - C4.1.1 à C4.3.3
	1.0 - 13 août 2026
	7 compétences documentées et vérifiables

Mise en situation réelle et fictive déclarée - preuves anonymisées et reproductibles

Sommaire

Sommaire	2
Déclaration de portée	3
1. Résumé exécutif	4
2. Correspondance avec le référentiel	5
3. Contexte technique et responsabilités	6
3.1 Architecture maintenue	6
3.2 Rôles	6
4. C4.1.1 - Gérer les mises à jour des dépendances	7
4.1 Processus précis	7
4.2 Mise à jour réalisée	7
4.3 Sécurisation	7
5. C4.1.2 - Concevoir la supervision et l'alerte	8
5.1 Sondes et finalité	8
5.2 Signalement	8
5.3 Résultat observé et exercice	8
6. C4.2.1 - Consigner les anomalies	9
6.1 Collecte	9
6.2 Anomalie rencontrée	9
7. C4.2.2 - Créer et déployer un correctif via CI/CD	10
7.1 Correctif déployé	10
7.2 Correctif d'accessibilité	10
7.3 Retour arrière	10
8. C4.3.1 - Proposer des améliorations	11
9. C4.3.2 - Établir le journal des versions	12
10. C4.3.3 - Collaborer avec le support	13
11. Protection des données et sécurité des preuves	14
12. Conclusion	15
13. Index des annexes	16

Certification : Expert en développement logiciel

Projet : Spity, réseau social pour la communauté de l'escalade

Candidat : Dorian Joly

Version du dossier : 1.0

Date : 13 août 2026

Référentiel : Ynov 2024, pages 15 à 17

Déclaration de portée

Ce dossier présente la maintenance de Spity, application Next.js 16 avec MariaDB, conteneurs Docker et GitHub Actions. Il couvre la gestion des dépendances, la supervision, la consignation et la correction des anomalies, les recommandations, le journal des versions et la collaboration support.

Les preuves sont de trois natures : sources versionnées, états externes publics figés en JSON et mise en situation fictive autorisée par le référentiel. Les simulations sont toujours annoncées comme telles. Aucun incident réel, échange humain ou déploiement n'est inventé. Aucun LXC n'a été utilisé pour constituer le dossier.

1. Résumé exécutif

Spity dispose d'une chaîne de maintenance reproductible : Dependabot surveille chaque semaine npm et GitHub Actions, la CI bloque lint, types, tests, audit, build, MariaDB, accessibilité, Lighthouse et recettes Playwright, tandis qu'une sonde externe vérifie la production. Les images de release sont immuables et associées à une version, une révision et un manifeste.

Le 13 août 2026, l'état vérifié est le suivant :

- 0 vulnérabilité de production et 0 alerte haute/critique dans l'audit complet après mise à jour des outils ;
- 152 tests Jest, 8 tests de maintenance, 11 scénarios MariaDB et 6 recettes Playwright ;
- 10 pages authentifiées sur 10 à 100 % Lighthouse accessibilité ;
- CI complète verte sur e3784b7 ;
- 29 exécutions de supervision historisées au moment de la collecte, avec production ok ;
- version observée en production : 0.1.0-jury, révision 49c4ea0 ;
- dérive de déploiement réelle consignée : la production reste 19 commits derrière la référence auditée.

La dérive n'a pas été corrigée par un déploiement non autorisé. Elle est transformée en action de release contrôlée, avec sauvegarde et retour arrière.

2. Correspondance avec le référentiel

C4.1.1	Processus précis : fréquence, périmètre, type	Cadence hebdomadaire et mensuelle, politique exécutable, audit planifié, SBOM, revue PR et lot réel qualifié	spity/MAINTENANCE.md, C411-01 à C411-03	Industrialisé et vérifié
C4.1.2	Supervision adaptée, sondes, critères qualité/performance, disponibilité	Route santé + MariaDB, contrôle 15 min, latence, retries, version/révision, artefact et issue automatique	spity/OBSERVABILITY.md, C412-01 à C412-03	Base fonctionnelle à approfondir
C4.2.1	Collecte structurée, fiche reproductible, analyse et préconisations	Formulaire incident, anomalie contraste complète et dérive production réelle	C421-01 et C421-02	Base fonctionnelle à approfondir
C4.2.2	Correctif décrit utilisant intégration/déploiement continu	Correctif sécurité observé en production, correctif accessibilité validé par quatre jobs CI et workflow Release	C422-01 et C422-02	Base fonctionnelle à approfondir
C4.3.1	Recommandations réalistes, argumentées, coûts/délais/gains	Cinq axes notés et chiffrés, priorisation et risques	C431-01	Base de pilotage à approfondir
C4.3.2	Journal des versions et correctifs déployés	Release v0.1.0, instance jury 0.1.0-jury, SHA et règle de tenue	C432-01, CHANGELOG.md	Base fonctionnelle à approfondir
C4.3.3	Problème résolu avec contexte, résolution et contributions	Mise en situation fictive support/mainteneur fondée sur une anomalie technique réelle	spity/SUPPORT.md, C433-01	Base fonctionnelle à approfondir

3. Contexte technique et responsabilités

3.1 Architecture maintenue

Le dépôt sépare l'application dans `spity/`, la CI/CD dans `.github/workflows/` et les documents dans `docs/`. L'application utilise Next.js 16.3, React 19, TypeScript, Drizzle ORM et MariaDB 11.4. Docker Compose isole l'application, la migration et la base. La production publique expose une route de santé sans secret.

La chaîne de livraison est structurée ainsi :

- modification du code ou du lockfile ;
- commit Git et push SSH ;
- qualité et recettes parallèles dans la CI ;
- construction d'images par SHA sur la branche d'intégration ;
- tag SemVer pour déclencher la release ;
- validation, smoke test, images candidates, manifeste et bundle ;
- promotion de production avec contrôle de version/révision.

3.2 Rôles

Le mainteneur est responsable de la qualification technique, des tests, des correctifs et du déploiement. GitHub Actions exécute les contrôles automatiques. Dependabot propose les mises à jour. Le product owner décide de la priorité métier et peut tenir le rôle support niveau 1 dans la configuration actuelle. Un utilisateur pilote valide les parcours lorsque ce rôle existe.

4. C4.1.1 - Gérer les mises à jour des dépendances

4.1 Processus précis

Dependabot analyse les dépendances npm chaque lundi à 06:00 et les actions GitHub à 06:30, fuseau Europe/Paris. Les mises à jour de production et de développement sont regroupées séparément et ciblent develop. Une revue manuelle mensuelle complète cette automatisation avec npm outdated, npm audit, les images Docker et les notes de version.

Les mises à jour de sécurité de production sont prioritaires. Les mineures compatibles sont groupées. Les majeures sont isolées avec analyse de rupture, recette ciblée et retour arrière. Le périmètre couvre npm, actions GitHub, Node.js, navigateurs CI et images ; il exclut toute modification automatique de secrets ou de données.

4.2 Mise à jour réalisée

L'audit du 13 août a trouvé des alertes hautes dans l'outillage Lighthouse/Puppeteer. Le lot a mis à niveau Lighthouse 12.6.1 vers 13.4.1, Puppeteer Core 24.43.1 vers 25.6.0, Playwright 1.61.1 vers 1.62.1, ESLint Next vers 16.3 et les types Node vers la branche 22.

Le résultat attendu est atteint : aucune alerte haute/critique dans l'audit complet et aucune vulnérabilité de production. Quatre alertes modérées restent liées à Drizzle/esbuild. La correction npm audit fix --force est rejetée car elle propose une rétrogradation cassante. La dérogation possède maintenant un propriétaire et une expiration, vérifiés automatiquement. La tentative de mise à jour de @hookform/resolvers a aussi révélé un conflit Ajv rendant le SBOM invalide ; la version 5.2.2 est verrouillée jusqu'au 13 octobre 2026 et le contrôle échoue si ce verrou ou son échéance dérive.

La commande npm run dependencies:check exécute les deux audits, applique dependency-policy.json, inventorie les versions en retard et produit un SBOM CycloneDX. Elle tourne chaque semaine dans dependency-maintenance.yml, conserve ses artefacts 90 jours et ouvre un ticket unique en cas d'échec. dependency-review.yml bloque en pull request toute nouvelle dépendance vulnérable de sévérité modérée ou supérieure. La preuve C411-03 conserve l'évaluation conforme sous Node 22, le SHA-256 du lockfile et les métadonnées du SBOM.

4.3 Sécurisation

Le lockfile garantit les versions transitives et son SHA-256 est figé dans la preuve. La CI rejoue l'ensemble des contrôles et bloque la promotion en cas d'échec. La procédure de retour arrière utilise les images immuables et la sauvegarde créée avant migration.

5. C4.1.2 - Concevoir la supervision et l'alerte

5.1 Sondes et finalité

La route `/api/health` confirme l'état de l'application, la connexion MariaDB, la version et la révision. Le script `check-health.mjs` ajoute un contrôle externe du statut HTTP, du JSON, de la latence et de la cohérence des métadonnées.

HTTP	2xx	Tout autre statut déclenche l'échec.
Application	status = ok	Toute dégradation est une alerte.
Métadonnées	version et révision non vides	Absence = binaire non traçable.
Performance	3 000 ms	Dépassement = dégradation.
Timeout	15 000 ms	Interruption bornée.
Résilience	2 reprises	Limite les faux positifs réseau.

Le workflow est planifié toutes les 15 minutes. GitHub peut retarder un cron ; la cible de détection n'est donc pas un engagement temps réel. L'objectif mensuel est 99,5 % de disponibilité.

5.2 Signalement

Chaque exécution conserve un rapport JSON 30 jours. En échec, une issue d'incident unique est créée ou mise à jour avec le run, le SHA et le rapport. À la récupération, un commentaire est ajouté et l'issue est fermée. L'issue reste distincte des données utilisateurs et ne contient aucun secret.

5.3 Résultat observé et exercice

La collecte publique a trouvé 29 runs de supervision, dont les plus récents sont réussis. La production répond ok. Un exercice local contrôlé reproduit un cas sain et un HTTP 503/applicatif degraded, vérifie deux tentatives et génère un rapport d'alerte sans toucher à la production.

6. C4.2.1 - Consigner les anomalies

6.1 Collecte

Le formulaire d'incident impose : date/fuseau, sévérité, version/révision, comportement observé, reproduction, attendu, preuves anonymisées, analyse, correctif, validation et clôture. Le formulaire support ajoute contexte utilisateur, priorité, escalade et contrôle de confidentialité.

6.2 Anomalie rencontrée

L'audit authentifié échouait à 0,96 sur certains états vides. Le problème était reproductible sur une base vierge et variait selon la surface claire ou sombre. La cause était un composant partagé qui héritait de couleurs non adaptées. La première correction par texte blanc déplaçait le défaut vers les cartes claires ; la cause racine a donc été corrigée en rendant le composant autonome.

La fiche C421-01 contient toutes les étapes, la cause, les préconisations, les commits et les résultats. Une seconde fiche consigne la dérive réelle de production : service disponible mais révision 19 commits derrière le code validé.

7. C4.2.2 - Créer et déployer un correctif via CI/CD

7.1 Correctif déployé

La production expose le commit 49c4ea0, intitulé `fix(security): update vulnerable runtime dependencies`. Il s'agit d'un correctif réel observé dans le binaire de production grâce à la route de santé. La preuve externe relie la révision, son message GitHub et la réponse publique.

7.2 Correctif d'accessibilité

Le traitement suit la chaîne actuelle : reproduction multi-environnement, trois commits atomiques, tests ciblés, qualité complète, push SSH et quatre jobs CI verts. La CI 31604246584 conserve cinq artefacts. La recette finale couvre six scénarios BC02 et dix pages authentifiées.

Ce correctif n'est pas présenté comme déjà déployé : la production expose encore 49c4ea0. La release future passera par les images candidates, le smoke test, le manifeste, les digests, le bundle et l'environnement protégé. Cette distinction évite de confondre « validé » et « déployé ».

7.3 Retour arrière

Le retour arrière remet le tag d'image précédent, vérifie la route de santé et conserve les journaux. Une restauration MariaDB n'est réalisée que si une migration empêche l'ancienne application de fonctionner, avec validation explicite et sauvegarde contrôlée.

8. C4.3.1 - Proposer des améliorations

Cinq recommandations sont chiffrées dans C431-01 : observabilité persistante, release alignée, couverture des API critiques, industrialisation du support et réduction de la dette Drizzle/esbuild.

La priorité opérationnelle est double : ajouter une rétention de métriques pour mesurer le service, puis promouvoir une release alignée avec main. Les gains attendus sont un MTTD inférieur à 15 minutes, une disponibilité mensuelle démontrable et la suppression de l'écart de 19 commits. Le coût initial est estimé à 3 à 5 jours.homme pour l'observabilité et 1 jour.homme pour la release hors surveillance.

Les propositions restent adaptées au projet : aucune refonte, un déploiement seulement avec autorisation, et une dette d'outillage traitée sur branche dédiée.

9. C4.3.2 - Établir le journal des versions

Le journal distingue ce qui est publié, observé et seulement candidat :

- release GitHub v0.1.0, publiée le 20 juillet 2026, commit cible 0bdd4e7 ;
- production 0.1.0-jury, révision observée 49c4ea0, correctif sécurité ;
- candidat e3784b7, CI verte mais non déployé, donc absent des versions déployées.

Chaque future entrée doit contenir tag, SHA, digests, migrations, correctifs, fonctionnalités, risques, rollback et preuve de santé. CHANGELOG.md décrit l'évolution produit ; le journal atteste le déploiement effectif.

10. C4.3.3 - Collaborer avec le support

La mise en situation fictive reprend l'anomalie de contraste. Le support niveau 1 simulé collecte le contexte d'une base vierge, reproduit les écrans, qualifie P2 et définit le critère de clôture. Le mainteneur reproduit sur plusieurs environnements, explique la cause racine, corrige le composant et fournit les preuves de CI.

La résolution est validée par le support simulé sur les parcours initiaux. La contribution du support est la précision du contexte et du résultat attendu ; celle du mainteneur est l'expertise technique, le correctif et la non-régression. L'enseignement commun est d'ajouter systématiquement l'état des données initiales aux tickets de rendu conditionnel.

Cette section ne prétend pas à un échange client réel. Elle répond à la modalité de mise en situation fictive du référentiel et fournit un exemple complet, reproductible et attribué.

11. Protection des données et sécurité des preuves

Les preuves contiennent uniquement des URLs publiques, SHA Git, versions, résultats de tests et comptes temporaires. Elles excluent `.env.local`, mots de passe, jetons, IP privées, exports MariaDB et données personnelles. Les rapports sont versionnés avec un manifeste SHA-256.

Les actions destructives restent interdites dans les procédures courantes : aucune suppression de volume, aucun `npm audit fix --force`, aucun déploiement sans sauvegarde et aucune réécriture de l'historique Git.

12. Conclusion

Les sept compétences disposent désormais d'une base documentée, de sources exécutables, d'états publics figés ou d'une mise en situation fictive déclarée. C4.1.1 est la première à franchir la définition renforcée de terminé : mécanisme réel, automatisation, cas d'échec testés, preuves reproductibles et exploitation décrite. Les six autres restent volontairement qualifiées comme bases à approfondir une par une.

Le principal risque ouvert n'est pas masqué : la production est saine mais en retard sur main. La prochaine action opérationnelle est une release versionnée autorisée, pas un déploiement improvisé. Cette transparence garantit que le dossier décrit l'état réel du logiciel.

13. Index des annexes

A1	C4.1.1	spity/MAINTENANCE.md
A2	C4.1.1	preuves/B4-C411-01-audit-dependances-2026-08-13.json
A3	C4.1.1	preuves/B4-C411-02-decision-maintenance-2026-08-13.md
A4	C4.1.1	preuves/B4-C411-03-contrôle-dependances-2026-08-13.json
A5	C4.1.2	spity/OBSERVABILITY.md
A6	C4.1.2	preuves/B4-C412-01-historique-supervision-2026-08-13.json
A7	C4.1.2	preuves/B4-C412-02-santé-production-2026-08-13.json
A8	C4.1.2/C4.2.1	preuves/B4-C412-03-exercice-alerte-2026-08-13.json
A9	C4.2.1	preuves/B4-C421-01-fiche-anomalie-accessibilite-2026-08-13.md
A10	C4.2.1	preuves/B4-C421-02-anomalie-derive-production-2026-08-13.md
A11	C4.2.2	preuves/B4-C422-01-correctif-et-ci-2026-08-13.json
A12	C4.2.2	preuves/B4-C422-02-traitement-correctif-ci-cd-2026-08-13.md
A13	C4.3.1	preuves/B4-C431-01-recommandations-2026-08-13.md
A14	C4.3.2	preuves/B4-C432-01-journal-versions-deployees-2026-08-13.md
A15	C4.3.3	spity/SUPPORT.md et preuves/B4-C433-01-collaboration-support-2026-08-13.md
A16	Intégrité	preuves/MANIFEST.sha256

Les sources complémentaires sont `.github/workflows/ci.yml`, `release.yml`, `production-monitoring.yml`, `dependency-maintenance.yml`, `dependency-review.yml`, `dependabot.yml`, les deux formulaires d'issue, `CHANGELOG.md`, `spity/dependency-policy.json`, `spity/DEPLOYMENT.md` et le référentiel officiel archivé.